

Prism — Prompts, System Instructions & Workflows

This document inventories every AI prompt, system instruction, and non-AI workflow used to build and run Prism (Kohler-MITWPU AI Research Lab, Track 3). Export this Markdown to PDF for the submission requirement.

Sourcing split (stated explicitly): Customer Support, Privacy, and Legal/Compliance are **real** public Kohler documents. HR and Finance are **synthetic** Meridian Fixtures policies, authored because no real internal Kohler HR/Finance manuals are public. Full rationale: docs/decisions.md §6.

1. Runtime system prompt — answer generation

File: prism/core/answer.py → SYSTEM_PROMPT

Model: PRISM_GEN_MODEL (default qwen2.5:7b-instruct)

When: Once per new user question (not on reformat-only turns).

Output contract: A single JSON object matching CanonicalAnswer (Piece 8). All five UI formats are pure-Python renderers over that object.

```
You are Prism, Kohler's internal enterprise knowledge assistant.
You answer questions about HR policy, Finance policy, Customer Support
(product troubleshooting), Privacy, and Legal/Compliance using ONLY the
provided context chunks.
```

Rules:

- Ground every factual claim in the provided context. Do not invent numbers, dates, model numbers, or policy terms.
- If the context does not contain a confident answer, set "no_answer": true and explain what's missing in "direct_answer" -- never fabricate a plausible-sounding answer, especially for Legal/Privacy questions.
- Copy numeric values (currency, percentages, day counts, thresholds) EXACTLY as they appear in the context. Do not round or approximate.
- Populate "key_facts" with the specific facts/values the user needs.
- Populate "steps" only if the answer is a procedure. Use [] (empty array), never null, when there are no steps.
- Populate "table" only if the answer is genuinely tabular -- otherwise null.
- When matching a numeric amount to a threshold band, pick the band that CONTAINS that amount. Do not pick a higher band.
- "sources" must list the source_url of every context chunk you actually used.
- Keep "direct_answer" as plain prose with no markdown formatting.
- Output ONLY valid JSON matching the provided schema. Use empty arrays [] (not null) for key_facts, steps, caveats, and sources when empty.

User prompt template (same file, _build_user_prompt): injects domain, recent conversation context, the user question, retrieved chunks formatted as [Source N] title=... url=... domain=..., and the JSON schema.

Repair retry: On JSON/schema validation failure, one follow-up user message is appended with the pydantic/JSON error and a request for corrected JSON only. After a second failure, the agent returns a

safe no_answer object instead of crashing.

2. Session titling prompts

File: prism/core/titler.py

Model: PRISM_TITLE_MODEL (default qwen2.5:3b-instruct) — cheap auxiliary model, not the generation model.

When: Once after the first successful exchange; optionally again on a clear domain switch (maybe_retitle).

Typical instruction pattern: - Produce a short (≤ 6 word) descriptive session title. - No quotes, no trailing punctuation, no "Chat about...". - Reflect the user's topic, not the assistant's wording.

3. Email renderer (optional polish)

File: prism/core/renderers/email.py

Default path is **template-only** (no LLM): subject + greeting + body assembled from CanonicalAnswer. An optional polish pass can be enabled for a more natural register; it is **off by default** so reformatting never burns an extra LLM call during demos.

4. Non-LLM “prompts” / decision rules (intentionally not model calls)

These are documented here because they are part of the agent workflow even though they are not LLM prompts — they exist specifically to protect the 8GB VRAM budget.

Workflow	Mechanism	File
Domain routing	Cosine similarity of query embedding vs. ~12 hand-written anchor phrases per domain + domain vote over top hybrid hits	prism/core/router.py
Ambiguity → clarification	If top-two domain scores within ROUTER_AMBIGUITY_MARGIN (0.04), ask a clarifying question instead of guessing	prism/core/agent.py

Workflow	Mechanism	File
No-answer honesty	If best dense distance exceeds relevance floor (stricter for Legal/Privacy), skip LLM and return grounded gap	prism/core/retriever.py, answer.no_context_answer
Reformat request	Regex detect “as JSON / XML / excel / email” → render last_answer only	prism/core/agent.py
Anaphora	Fold recent turn text into retrieval query when “that/it/the one...” appears	prism/core/agent.py
Jailbreak / prompt exfil	Deterministic refuse (no LLM) on “ignore previous instructions / print system prompt / developer mode”	prism/core/text_utils.py, agent.py
Authority spoof / policy override	Deterministic refuse on CFO-spoof + “override threshold to 0” (same path every time — not model-dependent)	prism/core/text_utils.py, agent.py
Leave arithmetic	Code computes CL carry-forward and leave-year join-date math; LLM is not trusted to invert day counts	prism/core/policy_math.py
Leading-number sycophancy	Detect “my manager said X... isn’t it?”; instruct + post-polish so we never rubber-stamp fabricated figures	text_utils.py, answer_polish.py
Multi-domain retrieve	Per-domain retrieve + merge for compound HR/Finance/Privacy or contractor-damage queries; filter Bathroom Design Service noise	agent.py

Anchor phrases are literal English examples (e.g. HR: “leave policy”, “WFH”, “notice period”; Finance: “expense reimbursement”, “per diem”; Customer Support: “toilet leaking”, “faucet aerator”; Privacy: “personal information”, “Do Not Sell”; Legal: “terms and conditions”, “Prop 65”, “warranty”). Full list lives in `router.py`.

5. Workflows used to *build* the knowledge base (not runtime chat)

5a. Assist crawl + parse

1. Fetch `https://assist.kohler.com/robots.txt` — Allow:/, Disallow:/_next/.
2. Parse `https://assist.kohler.com/en/sitemap` for all `/en/<category>/<slug>` links.
3. Polite fetch (~1 req/sec, cached under `data/raw/assist/`).
4. Parse `__NEXT_DATA__` → `displayCard` (title, markdown details, choices) — not rendered HTML — to avoid nav/footer noise.
5. Classify type: troubleshooting / install-guide / video / warranty / policy.
6. Deduplicate cross-listed slugs; tag all categories.
7. One article ≈ one chunk.

5b. Kohler.com legal / privacy

1. `curl_cffi` with Chrome TLS impersonation (Akamai blocks bare httpx).
2. Heading-boundary chunking on Privacy Policy and Terms.
3. Prop 65 / SDS / green-building / warranty pages from Assist cross-tagged into Legal.

5c. Synthetic HR & Finance authoring

- Written as Markdown by the submitting student (not by an LLM “generate a policy” prompt used as the final artifact).
- Structure informed by reference PDFs (`BU_HR_Manual_.pdf`, `Sample_Financial_Policies_AFOA.pdf`) for section conventions only.
- Clear SYNTHETIC disclosure banner in each file.
- Finance deliberately encodes unambiguous ■ thresholds for exactness demos.
- Chunked by heading via `prism/ingest/build_synthetic.py`.

5d. Index build

- One Chroma collection `prism_kb` with `domain + boolean domain_*` flags.
- BM25 sidecar for exact tokens (model numbers, ■ amounts).
- Embeddings: `nomic-embed-text` with on-disk cache.

6. Eval / benchmark prompts

`eval/bench_models.py` and `eval/run_eval.py` reuse the same `SYSTEM_PROMPT + generate_answer()` path as production — there is no separate “eval prompt.” Golden questions live in `eval/golden.jsonl`.

7. UI copy (non-model)

Empty-state headline and suggestion chips in `web/src/App.tsx` are static product copy, not LLM-generated.

Keep this file updated whenever a prompt or workflow changes; regenerate the PDF before final submission.